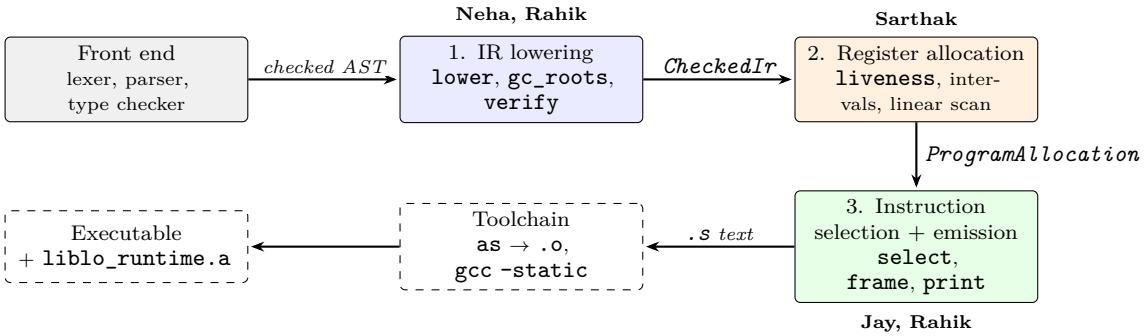
P2 Design Document: Native Code Generation (LO → x86-64)

Neha Reddy Sarthak Dayal Jay Shim Rahik Sikder
CS 378H, Fall 2026

1 Architecture at a Glance

The compiler is a multi-pass ahead-of-time pipeline. Every arrow below is a typed hand-off, so each stage can be tested and dumped on its own (`-dump-ir`, `-emit-asm`, `-native`). Our work is three parts (numbered); assembling and linking are done by the system toolchain.



Names, field offsets, and vtable layouts live in shared modules (`layout.rs`, `symbols.rs`, `ir/runtime.rs`, `ir/statics.rs`), so lowering and codegen cannot disagree about them. The allocator hands codegen only a map (virtual register ↦ register or spill slot), a spill count, and the callee-saved registers used. A trivial `-spill-all` allocator produces the same map type, which gave us a correct baseline to diff against.

2 IR and Lowering

Why an IR. P1's single-pass WebAssembly emitter wrote bytes as it walked the AST. It could not look backward or forward, so it could not reorder code, reuse a value, or decide where a value lives. WASM hid that cost behind its operand stack and locals. Native code needs a register assignment, which is a whole-function decision made after the whole function exists. The IR is the object that decision is made on.

Model. A `FunctionIr` is a list of basic blocks with a single entry. Each block is straight-line instructions followed by exactly one terminator (`br`, `cbr`, `ret`, `abort`). Values are mutable virtual registers (not SSA), each typed `Int32`, `Bool`, `Ref` (collectible heap pointer or static ref), `Ptr`, or `CodePtr(sig)`. Keeping `Ref` distinct from `Ptr` lets the GC pass find roots without re-deriving types. Instructions are explicit three-address forms: `Copy`, `Binary`, `Unary`, `Load [base+off]`, `Store`, `Call`, `CallIndirect`, `RootStore`, `RootLoad`. Operands are virtual registers, immediates, or symbols. A `ProgramIr` also carries symbols, interned signatures, and data definitions for `.rodata`/`.data`.

Lowering is syntax-directed: `Lowerer::expression` returns the `Operand` holding the expression's value and appends instructions as a side effect, while `statements` builds blocks and wires terminators. Runtime obligations are emitted as explicit IR so that they can be verified rather than assumed.

Source construct	Lowered form
`new C(args)`	`call lo_alloc(@class_C)`; store `LO_EMPTY_STRING` into each String field (inherited ones included); call the constructor
`r.m(args)`	`r == null` branches to a block ending in `abort lo_abort_null_receiver`; otherwise load descriptor `[r+0]`, vtable `[desc+48]`, slot `[vtable+8k]`, then `call_indirect`. No guard for `this.m()` or `super.m()`, since `this` is never null; every other receiver, including `in`/`out`/`err`, keeps it
`o.f = v` (v a reference)	`store [o+off], v` followed by `call lo_gc_write_barrier(o, off, v)`
`a / b`, `a % b`	LO division is total: $x/0 = -1$, $x\%0 = x$, $x/-1 = -x$ (wrapping), $x\%-1 = 0$. Divisors 0 and -1 are branched around `idiv` in IR (resolved statically for constant divisors), so it can trap neither on 0 nor on INT_MIN/-1
constructors	parent constructor call first, then field initializers

`CheckedIr::verify` enforces these contracts before any backend runs: types, terminators, signatures, symbols, definite assignment, and the write-barrier rule (every pointer store has a barrier, except stores into `lo_bindings`).

Sample dump (`class_basic.lo`: `b = new Box(); b.set(7);`, trimmed). The root traffic is explained in §5.

```
func @lo_method_4_Main_4_main(this:Ref) -> Int32 entry .L0 roots 3 {
.L0:
  t2:Ref = call @lo_alloc(@lo_class_3_Box)
  root_store root0, t2
  call @lo_ctor_3_Box_0(t2)
  t2:Ref = root_load root0           ; reload: a moving GC may have relocated it
  b:Ref = t2
  t3:Bool = b == null
  cbr t3, .L1, .L2
.L1:
  abort @lo_abort_null_receiver(@lo_bytes_0, 3)    ; bytes "set"
.L2:
  t4:Ptr = load [b + 0]              ; class descriptor
  t5:Ptr = load [t4 + 48]            ; vtable
  t6:CodePtr(Ref, Int32) -> Void = load [t5 + 8]    ; slot 1 = set
  root_store root1, b
  call_indirect t6(b, 7)
  b:Ref = root_load root1
```

3 Instruction Selection

Strategy. Selection is a single forward pass (`select_function`) with one fixed pattern per IR instruction kind. It is not BURS or dynamic-programming tiling: with a ∼40-instruction target subset and an IR whose instructions are already one machine-sized operation, a minimal-cost tiler would find almost nothing to merge. The payoff is auditability. Every emitted `Inst` is checked against the permitted subset by `Inst::check`, including the hand-built `main` wrapper.

Reserved scratch registers. `rax, rdx, r10, r11` are never allocated (along with `rsp, rbp`). `rax` is the accumulator and return register, `rdx` is overwritten by `cdq`/`idiv`, and `r10`/`r11` are scratch for spilled operands, call targets, and parallel-move cycles. This makes every fixup local: no pattern needs a temporary from the allocator or has to evict a live value.

Two-address repair. For $t_2 = t_0 \oplus t_1$ the selector computes in `rax`: `mov eax, t0; add eax, t1; mov t2, eax`. A spilled right operand is first loaded through `r10`, so at most one operand touches memory. Comparisons are `cmp; sete al; movzx eax, al`.

Division. `idiv` pins `edx:eax`. The selector emits `mov r10d, divisor; cdq; idiv r10d`, so the divisor cannot alias `eax` or `edx`, then reads the quotient from `eax` or the remainder from `edx`. Both are reserved, so nothing live is clobbered.

Widths. `Int32` and `Bool` use 32-bit operations; `Ref`, `Ptr`, and `CodePtr` use 64-bit. Width comes from the destination's `IrType`. Booleans are zero-extended with `movzx eax, al`, and a call's `bool` result is re-normalized the same way because System V leaves the upper bits undefined. *Deviation from the brief:* there is no `movsxd`. Pointers are always 64-bit values and field offsets are compile-time immediates, so no runtime 32→64 sign extension arises. `lea` is used only to form RIP-relative symbol addresses, never as an adder.

Addressing and dispatch. Loads and stores use `[base + disp]` with displacements from `layout.rs`. Virtual dispatch is the double load (descriptor → vtable → slot) followed by `mov r11, fn; call r11`. Static data is addressed RIP-relative.

4 Liveness and Linear-Scan Allocation

Dataflow. Liveness is a backward analysis iterated to a fixed point over the blocks in reverse order:

$$\mathrm{OUT}[B] = \bigcup_{S \in \mathrm{succ}(B)} \mathrm{IN}[S], \qquad \mathrm{IN}[B] = \mathrm{USE}[B] \cup \big(\mathrm{OUT}[B] \setminus \mathrm{DEF}[B]\big),$$

where USE[B] holds registers read before any definition in B. Registers are not SSA, so DEF acts as a kill per block.

Intervals. Blocks are linearized in layout order and every instruction and terminator receives a position. Each virtual register becomes *one* conservative interval $[\mathrm{start}, \mathrm{end}]$ (inclusive) from its first definition or live-in to its last use or live-out. This is safe across branches and loops, at the cost of filling in holes.

Allocation follows Poletto and Sarkar. Intervals are processed by increasing start. `expire_finished_intervals` frees registers whose interval ended. If a usable register is free, the interval takes the first one in pool order. Otherwise `spill_at_interval` compares against the active interval with the *furthest end* among those holding a usable register: if that one ends later it is evicted and its register reused, and otherwise the new interval is spilled. A spilled register lives

in its stack slot for its whole lifetime.

Registers and calling convention. The pool has ten registers, ordered `rsi, rdi, r8, r9, rcx` (caller-saved) then `rbx, r12-r15` (callee-saved). Reserved are `rsp, rbp` and the four scratch registers above. The calling-convention rule is applied *per value*: a value that is read after some call returns (`live_across_calls`) may use only callee-saved registers or a spill slot, while every other value may use any register, caller-saved first because those need no save or restore. In a function that makes calls, parameters also count as surviving a call, because `lo_push_frame` runs in the prologue after the parameter moves. A callee-saved register is pushed and restored only if it ends up in the final map. Because `gc_roots` reloads every `Ref` from its root slot after each call, a `Ref`'s pre-call register is never read again, so in practice only `Int32` and `Bool` values are constrained. This replaces call-site splitting and save/restore code.

Argument and return registers are not reserved, so allocated values can sit in `rdi`, `rsi`, `rcx`, `r8`, `r9`. Call setup therefore writes the register arguments with the same `moves.rs::parallel_move` used for incoming parameters, which orders the moves and breaks cycles through `r10`. An indirect call target is loaded into `r11` first, before any argument register is overwritten. Fixed-register constraints from `idiv` and `rax` results are served by the reserved set.

Linear scan versus graph coloring. Chaitin–Briggs builds an interference graph (up to $O(N^2)$ edges), then loops through simplify, potential-spill, select, and conservative coalescing, rebuilding the graph after each actual spill. It yields better code, especially through coalescing and live-range holes. Linear scan is $O(N \log N)$ with no rebuild loop, but the deciding factor was ease of implementation: we want to be able to read and understand all of the code we wrote, and linear scan is a single pass over sorted intervals that is much simpler than building, simplifying, and coalescing a graph. We checked correctness against `-spill-all`. This is a design argument; we did not implement or benchmark a graph-coloring allocator.

5 Stack Frames, GC Protocol, and Emission

Frame layout. `Frame` owns every offset; the selector never computes one.

```
[rbp + 16 + 8k]  stack argument k (7th onward)
[rbp + 8]        return address
[rbp]            saved rbp
[rbp - 8(i+1)]   saved callee-saved register i
                 ShadowFrame { parent, num_roots, roots[] }  (only if function calls)
                 spill slots, 8 bytes each
                 ret_save                                    (only if function calls)
                 padding: rsp = 0 (mod 16) at every call
```

Stack-passed arguments are pushed right to left with one pad slot when the count is odd, and cleaned up after the call.

GC safepoints. Every call may allocate, and a moving collector relocates objects, so a `Ref` held in a register is stale after any call. The protocol is split between IR and codegen:

1. `ir/gc_roots.rs` finds the `Ref`s live across each call (using the same liveness analysis) and inserts `root_store slot, v` before the call and `root_load` into the register after it. It also nulls the pass's other slots so stale pointers are never scanned.
2. The prologue builds the `ShadowFrame` in the native frame, zeroes `roots[]`, and calls `lo_push_frame`. The epilogue saves the return value, calls `lo_pop_frame`, and restores it. Functions with no calls skip all of this.

Because roots are ordinary IR instructions, the allocator needs no GC knowledge and `verify` can check the pass's output.

Emission and toolchain. `emit_program` produces `.text` (one function per `FunctionIr`, labels namespaced by function). `emit_data` produces `.rodata` (class descriptors, vtables, names, string literals) and `.data` (`lo_bindings`, the static root frame for `in`/`out`/`err`). Only `print.rs` formats assembly text, and that text is the compiler's final output. A synthesized `main` (`push rbp; mov rbp, rsp; call lo_runtime_init; call lo_entry; pop rbp; ret`) initializes the runtime and then calls the lowered IR function `lo_entry`. It is ordinary lowered IR, with no special prologue: it pushes the `lo_bindings` frame, allocates `in`/`out`/`err`, constructs `Main`, calls `Main.main` directly (`Main` cannot be extended and `lo_alloc` never returns null, so no dispatch or null check is needed), pops the frame, and returns the exit code in `eax`. `gcc -static` supplies `_start`, which passes `eax` to `exit`. We do not implement a linker. `assemble_and_link` writes the `.s` file, runs `as` to produce a `.o`, and lets `gcc -static` link it against `liblo_runtime.a`, so every `call lo_*` resolves to a direct call with no PLT or GOT.

6 Verification and Limitations

Both allocators are CLI-selectable (`-linear-scan` default, `-spill-all` baseline), so any program can be built both ways and checked against `tests/expected/`. **Limitations:** whole-interval spills; no splitting, coalescing, or peephole pass.